

Prog1 nyilvántartás programozói dokumentáció

A program célja:

A program a programozás alapjai 1 tárgyat felvevő tanulók, a kurzusok (előadások, gyakorlatok és laborok) és az oktatók adatait tárolja, illetve funkciói a tárgy követelményeire épülnek. A nyilvántartás a hallgatók neve és NEPTUN kódja mellett eredményeiket, hiányzásait tárolja, illetve a követelményeket ismerve meghatározza a tárgy pontszámát, a jegyet és hogy az adott hallgató teljesítette-e a tárgyat. A program nem kezeli külön az IMSC képzésben résztvevő hallgatókat, az általános követelményeket ismeri. A tárgyat nem lehet teljesíteni a megfelelő nagy házi feladat és ZH pont nélkül, ezért fontos funkciója a programnak a pótlásra szoruló hallgatók kilistázása. Emellett a program visszajelzésként és érdekességgként ranglistát állít fel a tárgybán szerzett pontszám vagy a szorgalmi feladatok eredménye alapján, valamint félév végén statisztikát készít a jegyekről, pontszámokról és a tárgyat teljesített hallgatók arányáról.

Új oktatót, kurzust és hallgatót lehet felvenni az adataik megadásával, illetve az oktatók, kurzusok és hallgatók adatai, a jelenlét, ZH, NHF pontszámok módosíthatóak. A program az adatokat kilépéskor fájlokban tárolja, a következő indításkor ezeket automatikusan betölti.

Adatszerkezetek választása

A program a tanárok adatait rendezetlen, strázsa nélküli láncolt listában tárolja, így egyszerűen lehet új oktatót felvenni. A tanár struktúra tartalmazza a tanár nevét, elérhetőségét, a hozzá tartozó kurzusok darabszámát, a tanárok kurzusait tároló rendezetlen, strázsa nélküli láncolt lista első elemére mutató pointert, illetve a tanárokat tartalmazó láncolt lista következő elemének memóriacímét.

```
typedef struct tanar{
    char nev[40];
    char elerhetoseg[30];
    int kurzus_db;
    struct kurzus_lista *kurzusok;
    struct tanar *kov;
} tanar;
```

```
typedef struct kurzus_lista{
    struct kurzus *kurzus_p;
    struct kurzus_lista *kov;
} kurzus_lista;
```

A kurzusok adatait rendezetlen, strázsa nélküli láncolt listában tárolja, így egyszerűen lehet új kurzust felvenni. A kurzus struktúra tartalmazza a kurzus egyedi kódját, időpontját, helyszínét, a kurzust tartó tanárra mutató pointert, a kurzus létszámát, a kurzust felvett hallgatókat tároló rendezetlen, strázsa nélküli láncolt lista első elemére mutató pointert, illetve a kurzusokat tartalmazó láncolt lista következő elemének memóriacímét.

```
typedef struct kurzus{
    char kurzus_kod[10];
    char idopont[9];
    char helyszin[10];
    tanar *oktato;
    int letszam;
    hallgato_lista *hallgatok;
    struct kurzus *kov;
} kurzus;
```

```
typedef struct hallgato_lista{
    hallgato *hallgato_p;
    struct hallgato_lista *kov;
} hallgato_lista;
```

Új kurzus felvételekor a kurzuskód generálását a program végzi. A kód labor esetén 'K', gyakorlat esetén 'G', előadás esetén 'E' karakterrel kezdődik, ezt követi a kurzus sorszáma. A sorszám kategóriák szerint külön értelmezett. Ehhez a program egy struktúrában tárolja a laborok, gyakorlatok és előadások aktuális darabszámát.

```
typedef struct kurzusok_db{
    int labor_db;
    int gyakorlat_db;
    int eloadas_db;
} kurzusok_db;
```

A program a hallgatók adatait rendezetlen, strázsa nélküli láncolt listában tárolja, így egyszerűen lehet új hallgatót felvenni. A hallgato struktúra tartalmazza a hallgató nevét, NEPTUN kódját, a hozzá tartozó 3 kurzusokra mutató mutatókat, a jelenléti adatokat tartalmazó sztringet, a kis ZH-k eredményeit tároló tömböt, a két ZH, IMSC és nagy házi feladat pontszámát, illetve a hallgatókat tartalmazó láncolt lista következő elemének memóriacímét.

```
typedef struct hallgato{
    char nev[40];
    char neptun[7];
    kurzus *labor;
    kurzus *gyakorlat;
    kurzus *eloadas;
    char jelenletek[15];
    int kisZH[6];
    int ZH1;
    int ZH2;
    int IMSC;
    int NHF;
    int szorgalmi;
    struct hallgato *kov;
} hallgato;
```

A ranglistát felállító funkcióban a program pontok struktúrában dolgozza fel a hallgatókat, a pontok struktúra tartalmazza a hallgatóra mutató pointert és a hozzá tartozó pontokat.

```
typedef struct pontok{
    hallgato *hallgato;
    int pont;
} pontok;
```

A neveket, elérhetőségeket, NEPTUN kódokat, kurzuskódokat, helyszíneket és időpontokat tároló sztringek maximális mérete a programban fix.

A tanar típus függvényei

`tanar *tanar_beszur(tanar *tanar_p, tanar *adat)`

Beszúr egy tanar típusú rendezetlen, strázsa nélküli láncolt lista elejére egy új elemet dinamikusán lefoglalt memóriaterületre. Első paramétere egy pointer, ami a lista elejére mutat. A második paramétere egy tanar objektum, amit a függvény lista elejére beszúr.

A függvény visszatérési értéke a beszúrás utáni láncolt lista elejére mutató pointer.

`tanar *tanar_beolvas(tanar *tanar_p, int *tanar_db)`

Beolvassa az "oktatok.csv" fájlt. Első paramétere egy láncolt lista első elemére mutató pointer, második paramétere az adatbázisban szereplő tanárok darabszáma memóriacímmel átadva.

Ha nem sikerül megnyitni a fájlt, "Nem lehet megnyitni a vezzerfajlt." üzenetet jelenít meg a képernyőn és a függvény NULL pointerrel tér vissza. Különben soronként beolvassa a fájlt, a tanar_beszur függvény használatával az első paraméterként kapott lista elejére beszúrja az adott sor adataiból előállított tanar objektumot, illetve sor beolvasása után növeli a második paraméterben kapott tanárok darabszámát. Miután összes sort beolvasta a fájlból a függvény visszatér az új oktatók beszúrása után létrejött láncolt lista elejére mutató pointerrel.

A függvény egy sorból maximum 330 karaktert olvas be, ez a becslés szerint elegendő egy legfeljebb 15 kurzussal rendelkező oktató adatainak beolvasásához.

`void oktato_modosit(tanar *tanar_p)`

Egy oktató kívánt mezőit módosító függvény. Paraméterben megkapja a tanárokat tartalmazó láncolt lista első elemére mutató pointert. A felhasználótól bekéri az oktató nevét és megjeleníti az oktatóról tárolt adatokat. Ha a felhasználó új adatot ír be, a program a tárolt adatokat az újonnan megadottakkal felülírja, ha a felhasználó csak Enter billentyűt üt le, az adatok nem módosulnak.

`tanar *tanar_listaz_beolvas(tanar *tanar_p)`

A függvény kilistázza az adatbázisban szereplő oktatók nevét, megjeleníti kurzusaik számát és beolvassa a konzolról a felhasználó által megadott nevet. Amíg a felhasználó adatbázisban nem szereplő nevet ad meg új név megadására szólít fel, majd visszatér a megadott nevű oktatóra mutató pointerrel.

`tanar *oktato_felv(tanar *tanar_ptr, int *tanar_db)`

Új oktatót vesz fel a képernyőről beolvasott adatok felhasználásával. Első paramétere egy láncolt lista első elemére mutató pointer, második paramétere az adatbázisban szereplő tanárok darabszáma memóriacímmel átadva.

Az oktató nevét és elérhetőségét a felhasználótól kéri be a konzolról, illetve az új tanar objektum kurzus_db mezőjét 0-ra, kurzusok mezőjét NULL pointerre állítja és az új tanar objektumot az első paraméterben kapott láncolt lista elejére beszúrja tanar_beszur függvényvel. Megnöveli a második paraméterben kapott tanárok darabszámát és visszatér a beszúrás utáni láncolt lista elejére mutató pointerrel.

`void felszabadit_tanar(tanar *tanar_ptr)`

Felszabadítja a paraméterben megkapott tanárokat tartalmazó láncolt listát.

`void mentes_tanar(tanar *tanar_ptr)`

A láncolt lista adatait "oktatok.csv" nevű fájlba menti. Első paramétere egy láncolt lista első elemére mutató pointer, a függvény ennek a láncolt listának adatait menti a fájlba.

Ha a fájlt nem sikerül megnyitni, "Nem sikerült megnyitni a fájlt." jelzést ír a képernyőre.

A kurzus típus függvényei

`kurzus *kurzus_beszur(kurzus *kurzus_p, kurzus *adat)`

Beszúr egy kurzus típusú rendezetlen, strázsa nélküli láncolt lista elejére egy új elemet dinamikusán lefoglalt memóriaterületre. Első paramétere egy pointer, ami a lista elejére mutat. A második paramétere egy kurzus objektum, amit a függvény lista elejére beszúr.

A függvény visszatérési értéke a beszúrás utáni láncolt lista elejére mutató pointer.

`kurzus_lista *kurzus_lista_beszur(kurzus_lista *eleje, kurzus *beszurt)`

Beszúr egy kurzus_lista típusú rendezetlen, strázsa nélküli láncolt lista elejére egy új elemet dinamikusán lefoglalt memóriaterületre. Első paramétere egy pointer, ami a lista elejére mutat. A második paramétere egy

kurzus objektum, amit a függvény lista elejére beszúr. A függvény visszatérési értéke a beszúrás utáni láncolt lista elejére mutató pointer.

```
kurzus_lista *kurzus_lista_torles(kurzus_lista *eleje, kurzus *torolt)
```

A függvény `kurzus_lista` típusú rendezetlen, strázsa nélküli láncolt listából töröl egy elemet. Az első paramétere egy pointer, ami a lista elejére mutat. A második paramétere egy kurzus objektum, amit a függvény töröl a listából. A visszatérési értéke a törlés utáni láncolt lista elejére mutató pointer.

```
kurzus *kurzus_beolvas(kurzus *kurzus_p, kurzusok_db *kurzus_db, tanar *tanar_ptr)
```

Beolvassa az "kurzus.csv" fájlt. Első paramétere egy láncolt lista első elemére mutató pointer, második paramétere az adatbázisban szereplő kurzusok darabszámát tároló objektum memóriacímével átadva, a harmadik egy tanárokat tartalmazó láncolt lista első elemére mutató pointer.

Ha nem sikerül megnyitni a fájlt, "Nem lehet megnyitni a vezetfajlt." üzenetet jelenít meg a képernyőn és a függvény NULL pointerrel tér vissza. Különböző soronként beolvassa a fájlt, megkeresi a harmadik paraméterben kapott listában azt az oktatót, amelyiknek a neve egyezik a kurzushoz beolvasott tanár nevével, majd a kurzus oktató mezőjét a megtalált tanár objektumra, ha nem található, akkor NULL pointerre állítja. Ha létezik a kurzushoz tartozó tanár, `kurzus_lista_beszur` függvénnyel a tanár kurzusok listájához adja az aktuális kurzust és a `kurzus_beszur` függvény használatával az első paraméterként kapott lista elejére beszúrja az adott sor adataiból előállított kurzus objektumot, illetve sor beolvasása után növeli a második paraméterben kapott kurzus objektum (a beolvasott kurzus kategóriájának) megfelelő mezőjének darabszámát. Miután összes sort beolvasta a fájlból a függvény visszatér az új kurzusok beszúrása után létrejött láncolt lista elejére mutató pointerrel.

A függvény egy sorból maximum 100 karaktert olvas be, ez a becslés szerint elegendő egy kurzus adatainak beolvasásához.

```
void kurzus_modosit(kurzus *kurzus_p, tanar *tanar_ptr)
```

Egy kurzus kívánt mezőit módosító függvény. Paraméterben megkapja a kurzusokat tartalmazó láncolt lista első elemére mutató pointert. A felhasználótól bekéri a kurzus kódját és megjeleníti a kurzusról tárolt adatokat. Ha a felhasználó új adatot ír be, a program a tárolt adatokat az újonnan megadottakkal felülírja, ha a felhasználó csak Enter billentyűt üt le, az adatok nem módosulnak. Ha a felhasználó a tanár nevét szeretné módosítani, de egy olyan nevet ad meg, ami nem szerepel az adatbázisban, a program kilistázza az adatokat és ismételten bekéri az adatbázisban létező nevet, majd a régi tanár kurzus_lista listájából törli a kurzust és az új tanár kurzus_listájához hozzáadja a kurzust, miközben a tanár objektumok kurzus_db értékét is módosítja.

```
kurzus *kurzus_felv(kurzus *kurzus_ptr, kurzusok_db *kurzus_db, tanar *tanar_ptr)
```

Új kurzust vesz fel a képernyőről beolvasott adatok felhasználásával. Első paramétere egy láncolt lista első elemére mutató pointer, második paramétere az adatbázisban szereplő kurzusok darabszámát tároló objektum memóriacímével átadva, a harmadik egy tanárokat tartalmazó láncolt lista első elemére mutató pointer.

Elsőként a felhasználónak választania kell előadás, gyakorlat, labor kategóriák között, majd ennek megfelelően a program legenerálja a kurzuskódot. Első karaktere a kurzus kategóriájára utal ('E', 'G', 'L'), ezután a kurzus sorszáma következik a kategóriáján belül.

Megnöveli a második paraméterben kapott kurzus objektum (a beolvasott kurzus kategóriájának) megfelelő mezőjének darabszámát.

A függvény megjeleníti a harmadik paraméterben kapott listában szereplő oktatókat, majd bekéri a felhasználó által választott oktató nevét és megkeresi a listában. Bekéri a felhasználótól a kurzus időpontját és helyszínét. A `letszam` mezőt 0-ra, a hallgatók mezőt NULL pointerre és a kurzus oktató mezőjét a megtalált tanár objektumra állítja. A `kurzus_beszur` függvény használatával az első paraméterként kapott lista elejére beszúrja a bekért adatokból előállított kurzus objektumot, a `kurzus_lista_beszur` függvénnyel a tanár kurzusok listájához adja az aktuális kurzust, a tanár objektum kurzus_db mezőjét megnöveli.

A függvény visszatér az új kurzus beszúrása után létrejött láncolt lista elejére mutató pointerrel.

```
void felszabadit_kurzus(kurzus *kurzus_ptr)
```

Felszabadítja a paraméterben megkapott kurzusokat tartalmazó láncolt listát.

```
void mentes_kurzus(kurzus *kurzus_ptr)
```

A láncolt lista adatait "kurzus.csv" nevű fájlba menti. Első paramétere egy láncolt lista első elemére mutató pointer, a függvény ennek a láncolt listának adatait menti a fájlba. Ha a fájlt nem sikerül megnyitni, "Nem sikerült megnyitni a fájlt." jelzést ír a képernyőre.

A hallgato típus függvényei

`hallgato_lista *hallgato_lista_beszur(hallgato_lista *eleje, hallgato *beszurt)`

Beszúr egy `hallgato_lista` típusú rendezetlen, strázsa nélküli láncolt lista elejére egy új elemet dinamikusán lefoglalt memóriaterületre. Első paramétere egy pointer, ami a lista elejére mutat. A második paramétere egy `hallgato` objektum, amit a függvény lista elejére beszúr.

A függvény visszatérési értéke a beszúrás utáni láncolt lista elejére mutató pointer.

`hallgato_lista *hallgato_lista_torles(hallgato_lista *eleje, hallgato *torolt)`

A függvény `hallgato_lista` típusú rendezetlen, strázsa nélküli láncolt listából töröl egy elemet. Az első paramétere egy pointer, ami a lista elejére mutat. A második paramétere egy `hallgato` objektum, amit a függvény töröl a listából. A visszatérési értéke a törlés utáni láncolt lista elejére mutató pointer.

`hallgato *hallgato_beszur(hallgato *hallgato_p, hallgato *beszurt)`

Beszúr egy `hallgato` típusú rendezetlen, strázsa nélküli láncolt lista elejére egy új elemet dinamikusán lefoglalt memóriaterületre. Első paramétere egy pointer, ami a lista elejére mutat. A második paramétere egy `hallgato` objektum, amit a függvény a lista elejére beszúr.

A függvény visszatérési értéke a beszúrás utáni láncolt lista elejére mutató pointer.

`hallgato *hallgato_beolvas(hallgato *hallgato_p, int *hallgato_db, kurzus *kurzus_p)`

Beolvassa az "hallgato.csv" fájlt. Első paramétere egy láncolt lista első elemére mutató pointer, második paramétere az adatbázisban szereplő hallgatók darabszáma memóriacímével átadva, harmadik paramétere egy kurzusokat tartalmazó láncolt lista első elemére mutató pointer.

Ha nem sikerül megnyitni a fájlt, "Nem lehet megnyitni a vezerefajlt." üzenetet jelenít meg a képernyőn és a függvény NULL pointerrel tér vissza. Különben soronként beolvassa a fájlt, a sor beolvasása után növeli a második paraméterben kapott hallgatók darabszámát, illetve `hallgato_beszur` függvény használatával az első paraméterként kapott lista elejére beszúrja a beolvasott sor adataiból előállított `hallgato` objektumot.

A harmadik paraméterben kapott listában megkeresi a hallgatóhoz tartozó kurzusokat, eltárolja a kurzus objektumok memóriacímét, a kurzusok `letszam` mezőjét megnöveli és `hallgato_lista_beszur` függvénnyel hozzáadja a hallgatót a kurzusok hallgatókat tartalmazó láncolt listájához.

Miután összes sort beolvasta a fájlból a függvény visszatér az új hallgatók beszúrása után létrejött láncolt lista elejére mutató pointerrel.

A függvény egy sorból maximum 180 karaktert olvas be, ez a becslés szerint elegendő egy hallgató adatainak beolvasásához.

`void hallgato_modosit(hallgato *hallgato_p, kurzus *kurzus_p)`

Egy `hallgato` kívánt mezőit módosító függvény. Paraméterben megkapja a hallgatókat tartalmazó láncolt lista első elemére mutató pointert. A felhasználótól bekéri a módosítani kívánt `hallgato` NEPTUN kódját és megjeleníti a hallgatóról tárolt adatokat. Ha a felhasználó új adatot ír be, a program a tárolt adatokat az újonnan megadottakkal felülírja, ha a felhasználó csak Enter billentyűt üt le, az adatok nem módosulnak. Ha a felhasználó a hallgató kurzus kódjának valamelyikét szeretné módosítani, bekéri az adatbázisban létező nevet, majd a régi kurzus `hallgato_lista` listájából törli a hallgatót és az új kurzus `hallgato_listajahoz` hozzáadja a hallgatót, miközben a kurzus objektumok `letszam` értékét is módosítja.

`hallgato *hallgato_felv(hallgato *hallgato_ptr, int *hallgato_db, kurzus *kurzus_ptr)`

Új kurzust vesz fel a képernyőről beolvasott adatok felhasználásával. Első paramétere egy láncolt lista első elemére mutató pointer, második paramétere az adatbázisban szereplő hallgatók darabszáma memóriacímével átadva, harmadik paramétere egy kurzusokat tartalmazó láncolt lista első elemére mutató pointer.

Új hallgató felvételekor a program a kis ZH, ZH, IMSC, szorgalmi pontokat 0-ra állítja be, illetve hiányzást tárol minden hétre. A felhasználó meg kell adja az új hallgató nevét, NEPTUN kódját. A `hallgato_beszur` függvény az első paraméterként kapott lista elejére beszúrja az adott sor adataiból előállított `hallgato` objektumot, illetve a program a `hallgato_felv` függvényben megnöveli program második paraméterben kapott hallgatók darabszámát. Kategóriánként megjeleníti a harmadik paraméterben kapott listában szereplő kurzusok kódját, a felhasználótól bekéri a választott kurzus kódját és a listában megkeresi a hallgatókhoz tartozó kurzusokat, a kurzusok `letszam` mezőjét megnöveli eltárolja a kurzus objektum memóriacímét, a `hallgato_lista_beszur` függvénnyel hozzáadja a hallgatót a kurzus hallgatókat tartalmazó láncolt listájához.

A függvény visszatér az új hallgatók beszúrása után létrejött láncolt lista elejére mutató pointerrel.

void felszabadit_hallgato (hallgato * hallgato_ptr)

Felszabadítja a paraméterben megkapott hallgatókat tartalmazó láncolt listát.

void mentes_hallgato(hallgato *hallgato_ptr)

A láncolt lista adatait "kurzus.csv" nevű fájlba menti. Első paramétere egy láncolt lista első elemére mutató pointer, a függvény ennek a láncolt listának adatait menti a fájlba. Ha a fájl nem sikerül megnyitni, "Nem sikerült megnyitni a fájlt." jelzést ír a képernyőre.

Jelenlétet, értékelést rögzítő függvények

void jelenlet_ertekeles(kurzus *kurzus_ptr, hallgato *hallgato_ptr)

A függvényben a felhasználó 5 lehetőség közül választhat, rögzíthet jelenlétet, ZH, NHF, szorgalmi, IMSC pontot. Paraméterben megkapja a kurzusokat és hallgatókat tartalmazó láncolt listák első elemére mutató pointerét, amik közül a megfelelőt átadja paraméterként a választás alapján meghívott a függvénynek.

void jelenlet(kurzus *kurzus_p)

A függvény jelenlétet rögzít egy kiválasztott kurzus hallgatóira. Paraméterben megkapja a kurzusokat tartalmazó láncolt lista első elemére mutató pointerét. Kिलistázza a kurzusokat és bekér a felhasználótól egy kurzuskódot, majd a hét sorszámát, amire a jelenlétet rögzíti. Ezután sorra megjeleníti a hallgatók nevét és NEPTUN kódját, akikhez a felhasználó 1 beírásával jelenlétet, 0 beírásával hiányzást rögzíthet, illetve Enter lenyomása esetén a már beírt adat vagy a kezdeti feltételezett hiányzás változatlan marad.

void ZH(kurzus *kurzus_p)

A függvényben a felhasználó 3 lehetőség közül választhat, rögzíthet kis ZH, első, illetve második nagy ZH pontszámot. Paraméterben megkapja a kurzusokat tartalmazó láncolt lista első elemére mutató pointerét. A listában megkeresi a felhasználó által megadott kurzusra mutató pointert, amit átad paraméterként a választás alapján meghívott a függvénynek.

void kis_ZH(kurzus *kurzus_p)

A függvény kis ZH pontszámot rögzít egy kurzus hallgatóira. Paraméterben megkapja a kurzusra mutató pointert, aminek hallgatóira a kis ZH adatokat rögzíti. Bekéri a rögzíteni kívánt kis ZH sorszámát, majd megjeleníti a hallgatók nevét és NEPTUN kódját, akikhez a felhasználó pontszámot rögzíthet, illetve Enter lenyomása esetén a már beírt adat vagy a kezdeti 0 pont változatlan marad.

void nagy_ZH(kurzus *kurzus_p, bool elso)

A függvény nagy ZH pontszámot rögzít egy kurzus hallgatóira. Első paraméterében megkapja a kurzusra mutató pointert, aminek hallgatóira a nagy ZH adatokat rögzíti, a második paraméter azt jelzi, hogy első vagy második nagy ZH adatai kerülnek rögzítésre, ha elso értéke true, az első nagy ZH, ha false, a második nagy ZH adatait rögzíti. A függvény megjeleníti a hallgatók nevét és NEPTUN kódját, akikhez a felhasználó ZH pontszámot rögzíthet, illetve Enter lenyomása esetén a már beírt adat vagy a kezdeti 0 pont változatlan marad.

void NHF(kurzus *kurzus_p)

A függvény NHF pontszámot rögzít egy kiválasztott kurzus hallgatóira. Paraméterben megkapja a kurzusokat tartalmazó láncolt lista első elemére mutató pointerét. Kिलistázza a kurzusokat és bekér a felhasználótól egy kurzuskódot. Ezután sorra megjeleníti a hallgatók nevét és NEPTUN kódját, akikhez a felhasználó NHF pontot rögzíthet, illetve Enter lenyomása esetén a már beírt adat vagy a kezdeti 0 pont változatlan marad.

void szorgalmi_IMSC(hallgato *hallgato_p, bool szorgalmi)

A függvény szorgalmi vagy IMSC pontszámot rögzít egy kurzus hallgatóira. Első paraméterében megkapja a hallgatókat tartalmazó láncolt listák első elemére mutató pointerét, aminek hallgatóira a pontszámot rögzíti, a második paraméter azt jelzi, hogy szorgalmi vagy IMSC pontok kerülnek rögzítésre, ha szorgalmi értéke true, a szorgalmi pontokat, ha false, az IMSC pontokat rögzíti. A függvény bekéri a hallgató nevét, majd a rögzíteni kívánt pontot, ami hozzáadódik a már tárolt pontszámokhoz (kezdetben minden hallgátónak 0 szorgalmi és IMSC pontja van).

Jegy számolásához szükséges függvények

int kisZH_pont(hallgato *hallgato_p)

A függvény paraméterben megkap egy hallgatóra mutató pointert és visszatér a hallgató 4 legjobb kis ZH pontjának összegével.

void jelenlet_db(hallgato *hallgato_p, int *labor, int *gyakorlat)

A függvény paraméterben megkap egy hallgatóra mutató pointert és cím szerint int típusú labor és gyakorlat változót, amiben megszámlolja a labor és gyakorlat jelenlétet.

int ZH_pont(hallgato *hallgato_p)

A függvény paraméterben megkap egy hallgatóra mutató pointert és visszatér a hallgató két ZH pontjának összegével vagy a második ZH pontjának kétszeresével, ha a második ZH pontszáma nagyobb, mint az első ZH pontszáma.

int jegy(hallgato *hallgato_p, int *targypontszam)

A függvény paraméterben megkap egy hallgatóra mutató pointert és cím szerint átveszi az int típusú targypontszam változót, aminek értékét a hallgató aktuális tárgy pontszámára módosít. A függvény visszatérési értéke a hallgató tárgy pontszáma, illetve a követelmények teljesülése szerinti jegye.

Kereső függvények

void kereses(kurzus *kurzus_ptr, hallgato *hallgato_ptr, tanar *tanar_ptr)

A függvényben a felhasználó három lehetőség közül választhat, kereshet oktatót, kurzust, illetve hallgatót, a választás alapján a függvény meghívja a megfelelő kereső függvényt. A függvény paraméterben megkapja a kurzusokat, hallgatókat és tanárokat tartalmazó láncolt listák első elemére mutató pointerét, amik közül a megfelelőt a kereső függvény meghívásakor átadja paraméterben.

void hallgato_keres(hallgato *hallgato_p)

A függvény egy hallgatót keres meg és kiírja adatait vagy jelzi, hogy nem található meg az adatbázisban. Paraméterként megkapja a hallgatókat tároló strázsa nélküli láncolt lista első elemére mutató pointerét. Bekér a felhasználótól egy NEPTUN kódot, ha a keresett hallgató megtalálható a listában, kiírja kurzusainak kódját, a labor és gyakorlat jelenlétet százalékos formában és az aktuális tárgy pontszámot, jegyet és a hallgató IMSC pontját. Ha a keresett hallgató nem található a listában, "A keresett hallgato nem talalhato az adatbazisban." üzenet ír. Mindkét esetben a továbblépéshez Enter lenyomását várja.

hallgato *hallgato_ptr_keres(hallgato *hallgato_p, char *neptun)

A függvény NEPTUN kód alapján megkeres egy hallgatót. Első paraméterében a hallgatókat tároló strázsa nélküli láncolt lista első elemére mutató pointerét, második paraméterében a keresett hallgató NEPTUN kódját kapja meg. A függvény visszatér a keresett hallgatóra mutató pointerrel vagy NULL pointerrel, ha a megadott NEPTUN kód nem található a listában.

void kurzus_keres(kurzus *kurzus_p)

A függvény egy kurzust keres meg és kiírja adatait vagy jelzi, hogy nem található meg az adatbázisban. Paraméterként megkapja a kurzusokat tároló strázsa nélküli láncolt lista első elemére mutató pointerét. Bekér a felhasználótól egy kurzuskódot, ha a keresett kurzus megtalálható a listában, kiírja az időpontját, helyszínét, a kurzust tartó oktató nevét és elérhetőségét. Ha a keresett kurzus nem található a listában, "A keresett kurzus nem talalhato az adatbazisban." üzenet ír. Mindkét esetben a továbblépéshez Enter lenyomását várja.

kurzus *kurzus_ptr_keres(kurzus *kurzus_p, char *kurzus_kod)

A függvény kurzuskód alapján megkeres egy kurzust. Első paraméterében a kurzusokat tároló strázsa nélküli láncolt lista első elemére mutató pointerét, második paraméterében a keresett kurzus kódját kapja meg. A függvény visszatér a keresett kurzusra mutató pointerrel vagy NULL pointerrel, ha a megadott kurzuskód nem található a listában.

void tanar_keres(tanar *tanar_p)

A függvény egy oktatót keres meg és kiírja adatait vagy jelzi, hogy nem található meg az adatbázisban. Paraméterként megkapja az oktatókat tároló strázsa nélküli láncolt lista első elemére mutató pointerét. Bekér a felhasználótól egy nevet, ha a keresett oktató megtalálható a listában, kiírja elérhetőségét és a hozzá tartozó kurzusok kódját. Ha a keresett oktató nem található a listában, "A keresett oktato nem talalhato az adatbazisban." üzenet ír. Mindkét esetben a továbblépéshez Enter lenyomását várja.

tanar *tanar_ptr_keres(tanar *tanar_p, char *nev)

A függvény név alapján megkeres egy oktatót. Első paraméterében az oktatókat tároló strázsa nélküli láncolt lista első elemére mutató pointerét, második paraméterében a keresett oktató nevét kapja meg. A függvény

visszatér a keresett tanarra mutató pointerrel vagy NULL pointerrel, ha a megadott név nem található a listában.

Teljesítményről visszajelző függvények

`void eredmény(hallgato *hallgato_p)`

A függvény megjeleníti az összes tanuló nevét, NEPTUN kódját, illetve a tárgypontszámot, jegyet, külön az IMSC pontokat is, illetve, hogy a hallgató teljesítette-e a tárgyat. A felhasználó Enter lenyomásával térhet vissza a főmenübe. Paramétere a hallgatókat tároló strázsa nélküli láncolt lista első elemére mutató pointer.

`void osszpont_rangsor(hallgato *hallgato_p, pontok *tomb)`

A függvény a második paraméterében cím szerint kapott pontok típusú tömbbe írja a hallgatókra mutató pointert és a hallgatók tárgy pontszámát. Első paramétere a hallgatókat tároló strázsa nélküli láncolt lista első elemére mutató pointer.

`void szorgalmi_rangsor(hallgato *hallgato_p, pontok *tomb, int *hallgato_db)`

A függvény a második paraméterében cím szerint kapott pontok típusú tömbbe írja a hallgatókra mutató pointert és a hallgatók szorgalmi pontszámát, amennyiben a hallgató szerzett szorgalmi pontot. Első paramétere a hallgatókat tároló strázsa nélküli láncolt lista első elemére mutató pointer, harmadik paramétere a hallgatók darabszáma cím szerint átadva, ez a függvény során módosul a szorgalmi ponttal rendelkező hallgatók számára.

`void rendez(pontok *tomb, int db)`

A függvény az első paraméterében cím szerint kapott pontok típusú tömb, a második paraméter a tömbben tárolt objektumok száma. A függvény a tömböt pontszám szerint csökkenő sorrendbe rendezi.

`void ranglista(hallgato *hallgato_p, int hallgato_db)`

A függvény ranglistát ír ki a hallgatókról összpontszám vagy szorgalmi pont szerint, a felhasználó választása szerint. (A szorgalmi pont szerinti ranglistában csak a szorgalmi pontot szerzett hallgatók jelennek meg). Első paramétere a hallgatókat tároló strázsa nélküli láncolt lista első elemére mutató pointer, második paramétere a hallgatók darabszáma. A függvény a hallgatók nevét és pontszámukat jeleníti meg, pontszám szerint csökkenő sorrendben. A felhasználó Enter lenyomásával térhet vissza a főmenübe.

`void statisztika(hallgato *hallgato_p, int hallgato_db)`

A függvény ranglistát jelenít meg a hallgatókról. Első paramétere a hallgatókat tároló strázsa nélküli láncolt lista első elemére mutató pointer, második paramétere a hallgatók darabszáma. Kiírja, melyik osztályzatból hány darab született, mennyi a jegyek, pontszámok átlaga, illetve a tárgyat teljesítő diákok aránya. A felhasználó Enter lenyomásával térhet vissza a főmenübe.

`void potlas(hallgato *hallgato_p)`

A függvény kiírja a NHF vagy a nagy ZH pótlásra szoruló hallgatók listáját a felhasználó választása szerint. Paramétere a hallgatókat tároló strázsa nélküli láncolt lista első elemére mutató pointer. A függvény a hallgatók nevét és NEPTUN kódját, NHF pótlás esetén a labor kódját és a laborvezető nevét, ZH pótlás esetén a gyakorlat kódját és a gyakorlatvezető nevét jeleníti meg. A felhasználó Enter lenyomásával térhet vissza a főmenübe.

Segéd függvények

`void fomenue_megjelenit()`

A főmenüben megjelenő 11 lehetőséget írja ki a képernyőre.

`void beolvas(char *str, int karakter_db)`

A függvény az első paraméterként átadott sztringbe beolvas egy sort a konzolról, legfeljebb a második paraméterben átadott karakter_db hosszúságban. Ha a beolvasott szöveg egyetlen új sor karakterből áll, nem módosul, különben a függvény levágja beolvasott szöveg végéről az új sor karaktert.

`bool ervenyes(char *kod, int lehetosegdb)`

Vizsgálja, hogy az első paraméterben kapott sztring egy érvényes kódot tárol-e. Az első paraméter a vizsgált sztring, a második az érvényes kódok darabszámát tartalmazza. Ha lehetosegdb 11, akkor a lehetséges kódok számot tartalmaznak 1-10-ig vagy 'x' karaktert. Ha lehetosegdb kisebb 11-nél, az érvényes kódok 1- Ha

lehetosegdb-ig terjedő számok. A függvény true értékkel tér vissza, ha kod egy érvényes kód, különben false értéket ad vissza.

bool uj inditas(char *szoveg)

A függvény két lehetőséget kínál a felhasználónak, amik közül kódbeírásával választhat. Az első lehetőséghez az első paraméterben kapott sztringet, a másodikhoz a "Vissza a fomenube" szöveget jeleníti meg. Ha a felhasználó az első lehetőséget választja a függvény true értékkel tér vissza, különben false értéket ad vissza.

void modositas(kurzus *kurzus_ptr, hallgato *hallgato_ptr, tanar *tanar_ptr)

A függvényben a felhasználó három lehetőség közül választhat, módosíthatja oktató, kurzus, illetve hallgató adatait, ez alapján a függvény meghívja a megfelelő módosításra szolgáló függvényt. A függvény paraméterben megkapja a kurzusokat, hallgatókat és tanárokat tartalmazó láncolt listák első elemére mutató pointerét, amik közül a megfelelőt a módosító függvény meghívásakor átadja paraméterben.

Az adatokat tartalmazó fájlok

A már megadott adatok a programból 'x' karakterrel való kilépéskor automatikusan 3 darab csv kiterjesztésű fájlba kerül mentésre. Ezeket a fájlokat a program a következő futtatáskor felhasználja. Az adott hallgatóhoz, kurzushoz és oktatóhoz tartozó adatokat egy sorban tartalmazzák "" karakterek között ',' karakterrel elválasztva. A neveket a vezetéknévet, keresztnévet és az esetleges harmadik nevet szóközzel elválasztva (pl.: "Kis Anna Dora"), ékezetmentes formátumban, az időpontokat "nap óra perc" formátumban, írja a fájlba, ahol a nap a hét napjának sorszáma (pl.: "1 15 15" hétfő 15 óra 15 percet jelöl).

A "hallgatok.csv" nevű fájl a hallgatók adatait tartalmazza: név, NEPTUN kód, a kurzusok kódja, jelenlét (0, 1 karakterek sorozata, ha jelen volt 1, ha hiányzott, 0), a kis ZH-k, 1. és 2. nagy ZH eredményei, NFH pont, IMSC pont, szorgalmi pont.

A "kurzusok.csv" nevű fájl a kurzusok adatait tárolja: kurzuskód, időpont, helyszín, oktató neve.

Az "oktatok.csv" nevű fájl az oktatók adatait tartalmazza: név, elérhetőség, tanított kurzusok száma, tanított kurzusok kódjai.